

PhytoGuard-Pi

Rapport de travail — Semaine 1

Membre C — Backend & Base de données

04 Mai 2026

Résumé de la journée

- ☑ Étape 1 — Base de données SQLite (schema.sql + init_db.py)
- ☑ Étape 2 — Serveur Flask avec 3 routes (app.py)
- ☑ Étape 3 — Règles agronomiques (agro_rules.json)
- ☑ Mise à jour — SEI + Stade phénologique conformément à l'énoncé
- ☑ Tests des 3 niveaux d'alerte validés (surveillance / intervention / critique)

Étape 1 — Base de données SQLite

La première tâche était de concevoir le schéma de la base de données qui stockera tous les diagnostics réalisés sur le terrain par le Raspberry Pi. La base utilise SQLite car elle fonctionne sans serveur — idéal pour un appareil embarqué.

Structure de la base — 3 tables

Table	Rôle dans le projet	Champs clés
parcelles	Enregistre chaque champ agricole	nom, culture, superficie, notes
diagnostics	Stocke chaque analyse de feuille	maladie, confiance, sévérité, produit, dose, recommandation
geolocalisations	Coordonnées GPS du diagnostic	latitude, longitude, altitude, précision

Fichier 1 — schema.sql

Ce fichier définit la structure complète de la base de données avec les 3 tables et les index :

```
-- PhytoGuard-Pi | schema.sql
-- Tables : parcelles, diagnostics, geolocalisations

PRAGMA foreign_keys = ON;

-- Table 1 : parcelles
CREATE TABLE IF NOT EXISTS parcelles (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  nom         TEXT      NOT NULL,
  culture     TEXT      NOT NULL,
  superficie  REAL,
  notes       TEXT,
  created_at  TEXT      DEFAULT (datetime('now'))
);

-- Table 2 : diagnostics
CREATE TABLE IF NOT EXISTS diagnostics (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  parcelle_id INTEGER REFERENCES parcelles(id) ON DELETE SET NULL,
  culture     TEXT      NOT NULL,
  maladie     TEXT      NOT NULL,
  confiance   REAL      NOT NULL,
  top3        TEXT,
  severite_bbch INTEGER,
  surface_lesions REAL,
  recommandation TEXT,
  produit     TEXT,
  dose_g_l    REAL,
  delai_recolte INTEGER,
  image_path  TEXT,
  rapport_pdf TEXT,
  created_at  TEXT      DEFAULT (datetime('now'))
);

-- Table 3 : geolocalisations
CREATE TABLE IF NOT EXISTS geolocalisations (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  diagnostic_id INTEGER NOT NULL REFERENCES diagnostics(id) ON DELETE CASCADE,
  latitude    REAL      NOT NULL,
  longitude   REAL      NOT NULL,
  altitude    REAL,
  precision_m REAL,
```

```

        created_at      TEXT      DEFAULT (datetime('now'))
    );

-- Index pour accélérer les recherches
CREATE INDEX IF NOT EXISTS idx_diagnostics_parcelle ON diagnostics(parcelle_id);
CREATE INDEX IF NOT EXISTS idx_diagnostics_maladie  ON diagnostics(maladie);
CREATE INDEX IF NOT EXISTS idx_diagnostics_date     ON diagnostics(created_at);

```

Fichier 2 — init_db.py

Ce script Python initialise la base de données à partir du schéma, et vérifie que les tables ont bien été créées :

```

# PhytoGuard-Pi | init_db.py

import sqlite3, os

DB_PATH      = "phytoguard.db"
SCHEMA_PATH  = "schema.sql"

def init_db():
    with open(SCHEMA_PATH, "r", encoding="utf-8") as f:
        schema = f.read()
    conn = sqlite3.connect(DB_PATH)
    conn.executescript(schema)
    conn.commit()
    conn.close()
    print(f"[OK] Base de données initialisée : {DB_PATH}")

def verify_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.execute("SELECT name FROM sqlite_master WHERE type='table'")
    tables = [row[0] for row in cursor.fetchall()]
    conn.close()
    print(f"[OK] Tables présentes : {sorted(tables)}")

if __name__ == "__main__":
    init_db()
    verify_db()

```

Résultat du test

Commande exécutée :

```
python init_db.py
```

Sortie obtenue :

```

[OK] Base de données initialisée : phytoguard.db
[OK] Tables présentes : ['diagnostics', 'geolocalisations', 'parcelles',
'sqlite_sequence']

```

Note : `sqlite_sequence` est une table interne gérée automatiquement par SQLite pour les AUTOINCRÉMENT. Sa présence est normale.

Étape 2 — Serveur Flask (app.py)

Flask est une bibliothèque Python qui permet de créer un serveur web léger. Dans PhytoGuard-Pi, ce serveur reçoit les résultats du modèle IA, les compare aux règles agronomiques, et les sauvegarde en base de données.

Les 3 routes de l'API

Méthode	Route	Rôle	Données reçues / retournées
GET	/	Vérifier que l'API tourne	Retourne : nom, version, heure, liste des routes
POST	/diagnose	Enregistrer un diagnostic	Reçoit : culture, maladie, confiance, sévérité, GPS
GET	/history	Consulter l'historique	Filtre par culture/maladie, retourne les diagnostics

Code complet — app.py

```
# PhytoGuard-Pi | app.py

import json, sqlite3
from datetime import datetime
from flask import Flask, request, jsonify

app = Flask(__name__)
DB_PATH = "phytoguard.db"
AGO_RULES_PATH = "agro_rules.json"

# Connexion à la base SQLite
def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

# Chargement des règles agronomiques
def load_agro_rules():
    try:
        with open(AGO_RULES_PATH, "r", encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}

# — Route 1 : GET / —————
@app.route("/", methods=["GET"])
def index():
    return jsonify({
        "projet" : "PhytoGuard-Pi",
        "version" : "0.1.0",
        "statut" : "en ligne",
        "heure" : datetime.now().isoformat()
    })

# — Route 2 : POST /diagnose —————
@app.route("/diagnose", methods=["POST"])
def diagnose():
    data = request.get_json(silent=True)

    # Validation des champs obligatoires
```

```

if not data:
    return jsonify({"erreur": "Corps JSON manquant"}), 400
for champ in ["culture", "maladie", "confiance"]:
    if champ not in data:
        return jsonify({"erreur": f"Champ manquant : {champ}"}), 400

culture      = data["culture"]
maladie      = data["maladie"]
confiance    = float(data["confiance"])
severite_bbch = data.get("severite_bbch", 0) or 0
surface_lesions = data.get("surface_lesions", 0) or 0

# Lecture des règles agronomiques
rules = load_agro_rules()
rule = rules.get(maladie, {}).get(culture) \
    or rules.get(maladie, {}).get("", {})

# Comparaison avec le SEI
sei = rule.get("sei", 10)
if severite_bbch >= 4 or surface_lesions >= sei * 3:
    recommandation = rule.get("recommandation_critique", "")
    niveau_alerte = "critique"
    alerte_sms = True
elif surface_lesions >= sei or severite_bbch >= 2:
    recommandation = rule.get("recommandation_sur_sei", "")
    niveau_alerte = "intervention"
    alerte_sms = False
else:
    recommandation = rule.get("recommandation_sous_sei", "")
    niveau_alerte = "surveillance"
    alerte_sms = False

# Sauvegarde en base SQLite
conn = get_db()
cursor = conn.execute(
    """INSERT INTO diagnostics
    (parcelle_id, culture, maladie, confiance, top3,
    severite_bbch, surface_lesions,
    recommandation, produit, dose_g_l, delai_recolte)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
    (data.get("parcelle_id"), culture, maladie, confiance,
    json.dumps(data.get("top3")), severite_bbch, surface_lesions,
    recommandation, rule.get("produit"),
    rule.get("dose_g_l"), rule.get("delai_recolte"))
)
# Géolocalisation si fournie
if data.get("latitude") and data.get("longitude"):
    conn.execute(
        "INSERT INTO geolocalisations VALUES
    (NULL, ?, ?, ?, NULL, NULL, datetime('now'))",
        (cursor.lastrowid, data["latitude"], data["longitude"])
    )
conn.commit(); conn.close()

return jsonify({
    "diagnostic_id"      : cursor.lastrowid,
    "niveau_alerte"      : niveau_alerte,
    "alerte_sms"         : alerte_sms,
    "recommandation"     : recommandation,
    "produit"            : rule.get("produit"),
    "dose_g_l"           : rule.get("dose_g_l"),
    "delai_recolte"      : rule.get("delai_recolte"),
}), 201

# — Route 3 : GET /history —————
@app.route("/history", methods=["GET"])
def history():
    limite = min(int(request.args.get("limite", 20)), 100)
    culture = request.args.get("culture")
    maladie = request.args.get("maladie")
    query = "SELECT * FROM diagnostics WHERE 1=1"
    params = []
    if culture: query += " AND culture = ?"; params.append(culture)

```

```
if maladie: query += " AND maladie = ?"; params.append(maladie)
query += " ORDER BY created_at DESC LIMIT ?"
params.append(limite)
conn = get_db()
rows = conn.execute(query, params).fetchall()
conn.close()
return jsonify({"total": len(rows), "diagnostics": [dict(r) for r in rows]})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Tests effectués

Test GET /

Commande :

```
| http://127.0.0.1:5000/ (navigateur)
```

Résultat :

```
| { "projet": "PhytoGuard-Pi", "statut": "en ligne", "version": "0.1.0" }
```

Test POST /diagnose

Commande :

```
| Invoke-WebRequest -Uri http://127.0.0.1:5000/diagnose -Method POST \
-ContentType "application/json" \
-Body '{"culture":"tomate","maladie":"mildiou","confiance":0.92}'
```

Résultat :

```
| StatusCode : 201 CREATED
{ "diagnostic_id": 1, "niveau_alerte": "surveillance", "alerte_sms": false,
  "produit": "Mancozèbe", "dose_g_l": 2.0, "delai_recolte": 7 }
```

Test GET /history

Commande :

```
| http://127.0.0.1:5000/history (navigateur)
```

Résultat :

```
| { "total": 1, "diagnostics": [
  { "id": 1, "culture": "tomate", "maladie": "mildiou",
    "confiance": 0.92, "created_at": "2026-05-04 17:01:54" }
  ] }
```

Étape 3 — Règles agronomiques (agro_rules.json)

Ce fichier JSON est le "cerveau agronomique" de PhytoGuard-Pi. Quand le modèle IA détecte une maladie, Flask consulte ce fichier pour déterminer la recommandation, le produit à appliquer, et le niveau d'alerte selon le SEI.

Structure du fichier

Champ	Type	Description
stade_phenologique	Texte	Moment critique de la saison pour cette maladie
sei	Nombre	Seuil Économique d'Intervention en % de surface atteinte
sei_unite	Texte	Unité de mesure du SEI (ex: % feuilles atteintes)
recommandation_sous_sei	Texte	Message si surface < SEI (surveiller à J+7)
recommandation_sur_sei	Texte	Message si surface ≥ SEI (traitement curatif)
recommandation_critique	Texte	Message si stade épidémique (alerte SMS)
produit	Texte	Nom du fongicide recommandé
dose_g_l	Nombre	Dose en grammes par litre
delai_recolte	Nombre	Jours à attendre avant la récolte après traitement
methode_application	Texte	Méthode d'application du produit

Exemple de règle — Mildiou sur Tomate

Voici un extrait du fichier agro_rules.json pour le mildiou sur tomate :

```
{
  "mildiou": {
    "tomate": {
      "stade_phenologique": "De la transplantation jusqu'a la nouaison des fruits",
      "sei": 10,
      "sei_unite": "% feuilles atteintes",
      "recommandation_sous_sei": "Surveiller a J+7. Verifier l'humidite.",
      "recommandation_sur_sei": "Traitement curatif. Eliminer les feuilles atteintes.",
      "recommandation_critique": "Stade epidemique - alerte SMS envoyee.",
      "produit": "Mancozeebe",
      "dose_g_l": 2.0,
      "delai_recolte": 7,
      "methode_application": "pulverisation foliaire"
    },
    "*": {
      "recommandation_sous_sei": "Surveiller a J+7.",
      "sei": 10,
      ...
    }
  }
}

-- Note : "*" est la regle par default si la culture n'est pas specifiee
```

Les 5 maladies couvertes

Maladie	Cultures couvertes	SEI (%)	Produit principal	Délai récolte
Mildiou	Tomate, Vigne, Pomme de terre	5 – 15	Mancozèbe	7 – 21 j
Oïdium	Vigne, Tomate, Blé	5 – 15	Soufre mouillable	3 – 35 j
Septoriose	Blé, Tomate	20 – 30	Propiconazole	7 – 35 j
Botrytis	Tomate, Vigne, Poivron	5	Iprodione	7 – 14 j
Rouille	Blé, Vigne	5 – 10	Tébuconazole	3 – 35 j

Logique SEI dans app.py

Flask compare automatiquement la sévérité détectée avec le SEI pour choisir le bon niveau d'alerte :

Condition	Niveau	alerte_sms	Message
surface < SEI ET bbch < 2	surveillance	false	Surveiller à J+7
surface ≥ SEI OU bbch ≥ 2	intervention	false	Traitement curatif obligatoire
surface ≥ SEI×3 OU bbch = 4	critique	TRUE ☐	Traitement urgent + SMS Twilio

Tests des 3 niveaux d'alerte

Test 1 — Niveau surveillance (surface < SEI)

```
Body: { "culture":"tomate", "maladie":"mildiou", "confiance":0.92 }
(surface_lesions non fournie = 0, en dessous du SEI=10)
```

```
{ "niveau_alerte": "surveillance", "alerte_sms": false,
  "recommandation": "Surveiller a J+7. Verifier l'humidite..." } ☒
```

Test 2 — Niveau intervention (surface ≥ SEI)

```
Body: { "culture":"tomate", "maladie":"mildiou", "confiance":0.92,
  "surface_lesions":25, "severite_bbch":2 }
(surface=25 >= SEI=10)
```

```
{ "niveau_alerte": "intervention", "alerte_sms": false,
  "recommandation": "Traitement curatif. Eliminer les feuilles atteintes." } ☒
```

Test 3 — Niveau critique (surface ≥ SEI×3 et bbch=4)

```
Body: { "culture":"tomate", "maladie":"mildiou", "confiance":0.92,
  "surface_lesions":35, "severite_bbch":4 }
(surface=35 >= SEI*3=30 ET bbch=4)
```

```
{ "niveau_alerte": "critique", "alerte_sms": true,
  "recommandation": "Stade epidemique - alerte SMS envoyee. Traitement urgent." } ☒
```


Bilan final — Semaine 1

Tâche	Fichier(s) produit(s)	Statut	Conforme à l'énoncé
Étape 1 — Base SQLite	schema.sql + init_db.py	☒ Testé	☒ Oui
Étape 2 — Serveur Flask	app.py	☒ Testé	☒ Oui
Étape 3 — Règles agro	agro_rules.json	☒ Testé	☒ Oui
SEI + Stade phénologique	agro_rules.json + app.py	☒ Testé	☒ Oui
3 niveaux d'alerte	app.py	☒ Testé	☒ Oui

Fichiers livrés

- schema.sql — structure de la base de données SQLite
- init_db.py — script d'initialisation de la base
- app.py — serveur Flask avec 3 routes et logique SEI
- agro_rules.json — règles agronomiques pour 5 maladies × plusieurs cultures
- phytoguard.db — base de données générée automatiquement